06: Memory-Mapped I/O

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University

Announcements

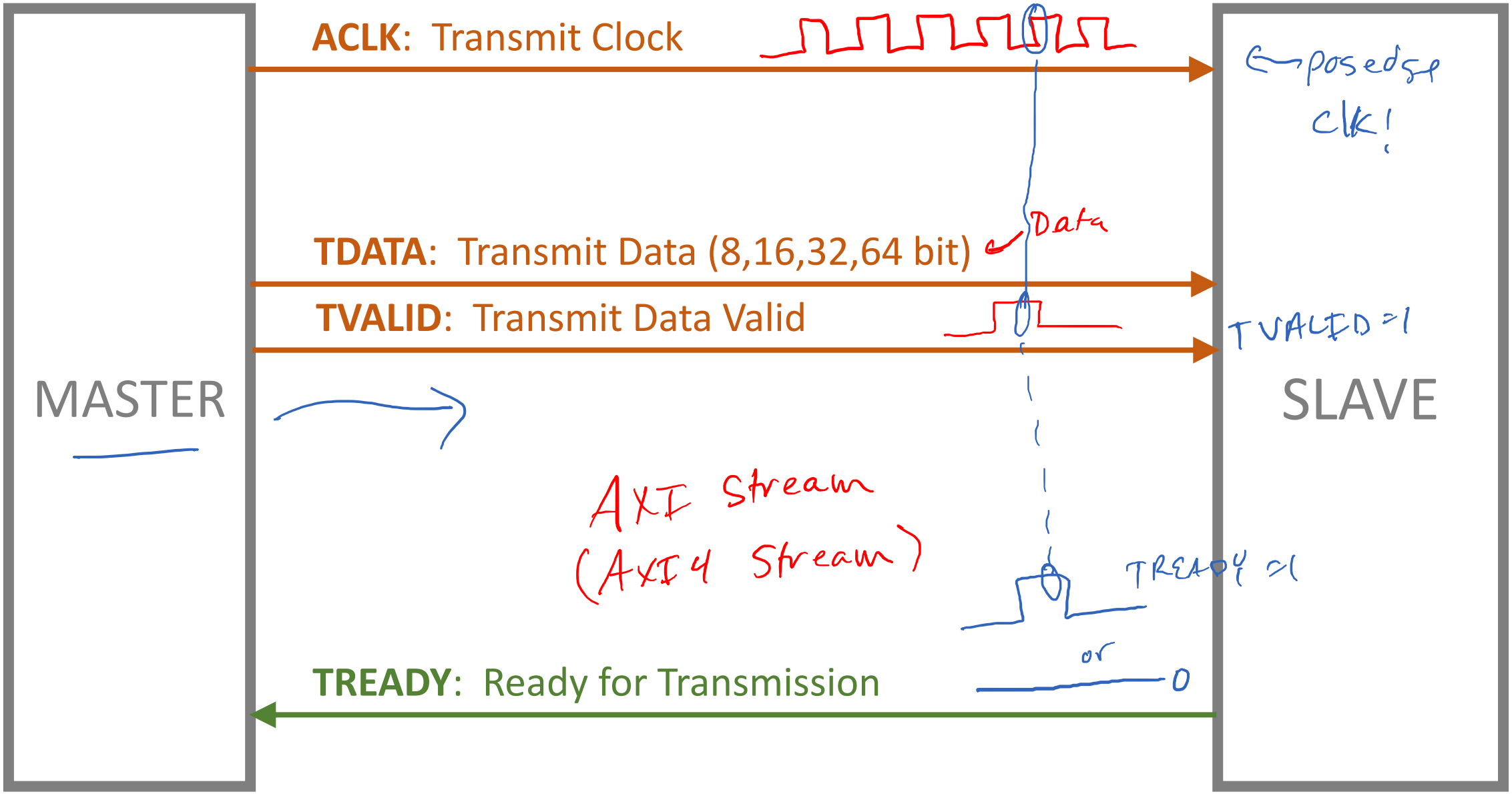
- P2 is live!
- Labs will continue to be virtual for now.
- Working on hardware box. Updates next week.

Optimizations thus far

- Algorithmic complexity
- Removing redundant computation
- Multithreading
- Multiprocessing*
- Python/C/Asm Interfacing

We could also map popcount to hardware

```
import cPopcount
print (cPopcount.cPopcount(0))

import hwPopcount
print hwPopcount.hwPopcount(0))
```


ACLK: Transmit Clock
posedge clk!
Data
TDATA: Transmit Data (8,16,32,64 bit)
TVALID: Transmit Data Valid
TVALID=1
MASTER
SLAVE
AXI Stream
(AXI4 Stream)
TREADY=1
or
0
TREADY: Ready for Transmission

Data is only transferred when

ACLK is on the positive edge **AND**

TVALID is **1** **AND**

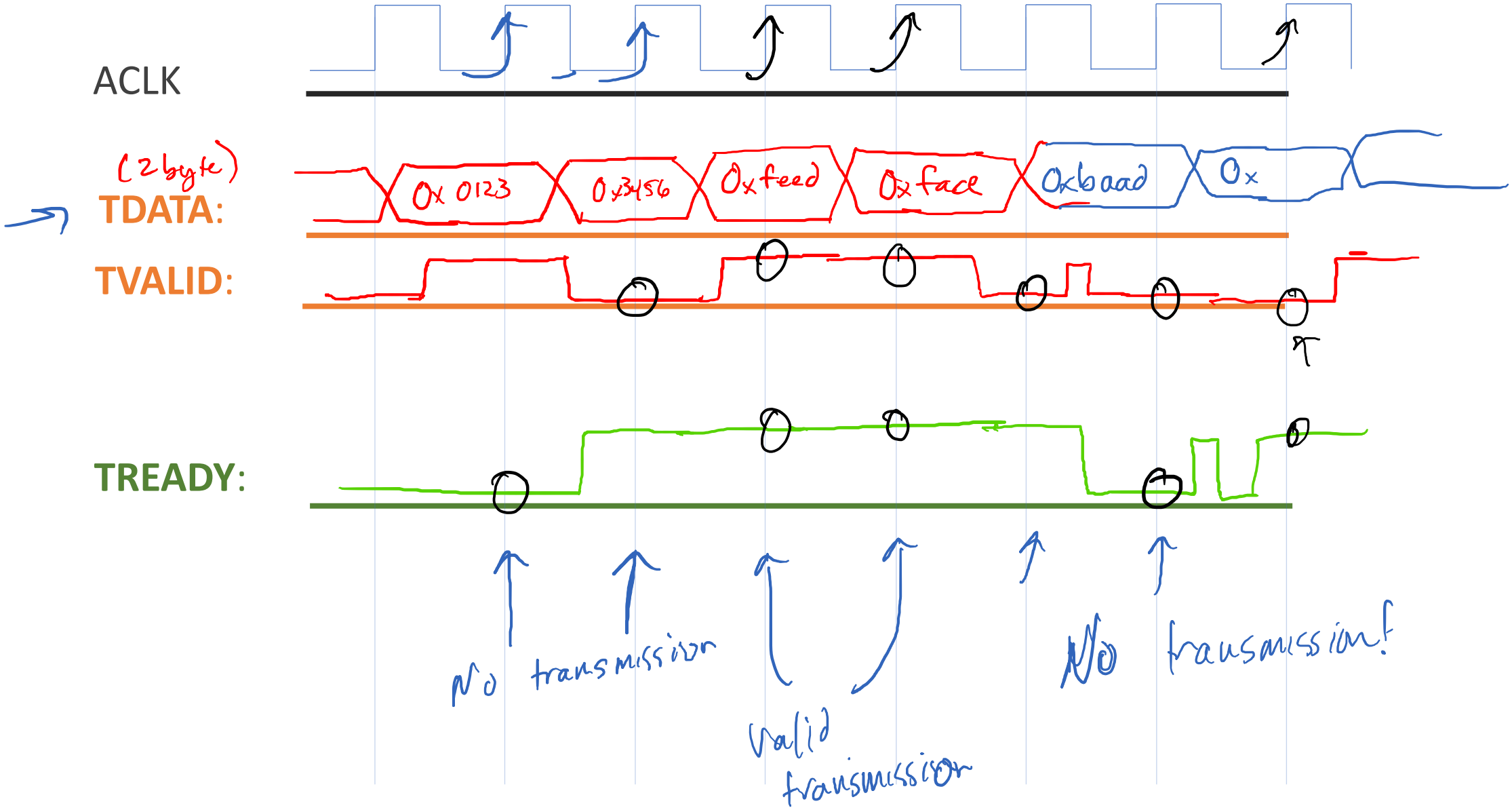
This indicates the **MASTER** is trying to transmit new data.

TREADY is **1.**

This indicates the **SLAVE** is ready to receive the data.

If either **TVALID** or **TREADY** are 0, no data is transmitted.

Transferring data on a AXI4-Stream Bus.



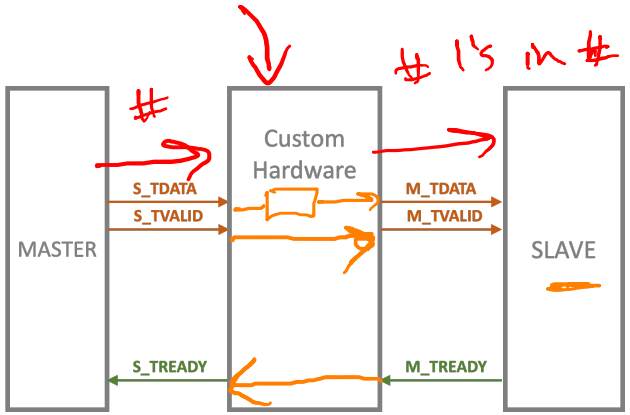
→ convert to decimal

Let's build popcount.

```
module custom_hw_popcount (
        input           ACLK,
        input           ARESET,
        input [31:0]    S_TDATA,
        input           S_TVALID,
        output          S_TREADY,
        output [31:0]   M_TDATA,
        output          M_TVALID,
        input           M_TREADY
);
```

we want to count

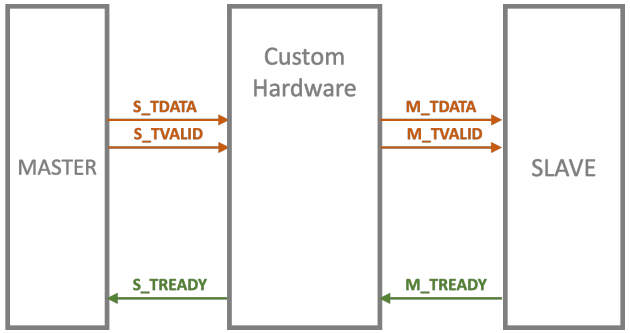
of 1's in S_TDATA




```
assign M_TDATA = S_TDATA[0] + S_TDATA[1] + S_TDATA[2] + ...
                 S_TDATA[30] + S_TDATA[31]

assign M_TVALID = S_TVALID;
endmodule
assign S_TREADY = M_TREADY;
```

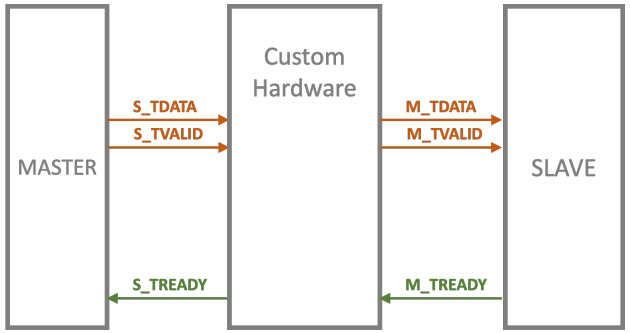
Let’s build popcount.

```
module custom_hw_popcount (
        input           ACLK,
        input           ARESETN,
        input [31:0]    S_TDATA,
        input           S_TVALID,
        output          S_TREADY,
        output [31:0]   M_TDATA,
        output          M_TVALID,
        input           M_TREADY
);

assign M_TDATA = S_TDATA[0] + S_TDATA[1] + S_TDATA[2] + … + S_TDATA[30] + S_TDATA[31];
assign M_TVALID = S_TVALID;
assign S_TREADY = M_TREADY;


endmodule
```



Let’s build popcount.

```
module custom_hw_popcount (
        input          ACLK,
        input          ARESETN,
        input [31:0]   S_TDATA,
        input          S_TVALID,
        output         S_TREADY,
        output [31:0]  M_TDATA,
        output         M_TVALID,
        input          M_TREADY
);

integer i;
reg [31:0] tmp;

always_comb begin
    tmp = 32’h0;
    for (i = 0; i < 32; ++i) tmp += S_TDATA[i];
end

assign M_TDATA = tmp;
assign M_TVALID = S_TVALID;
assign S_TREADY = M_TREADY;

endmodule
```



tmp = 0;

tmp = tmp + S_TDATA[0];

tmp = tmp + S_TDATA[i] ⇒

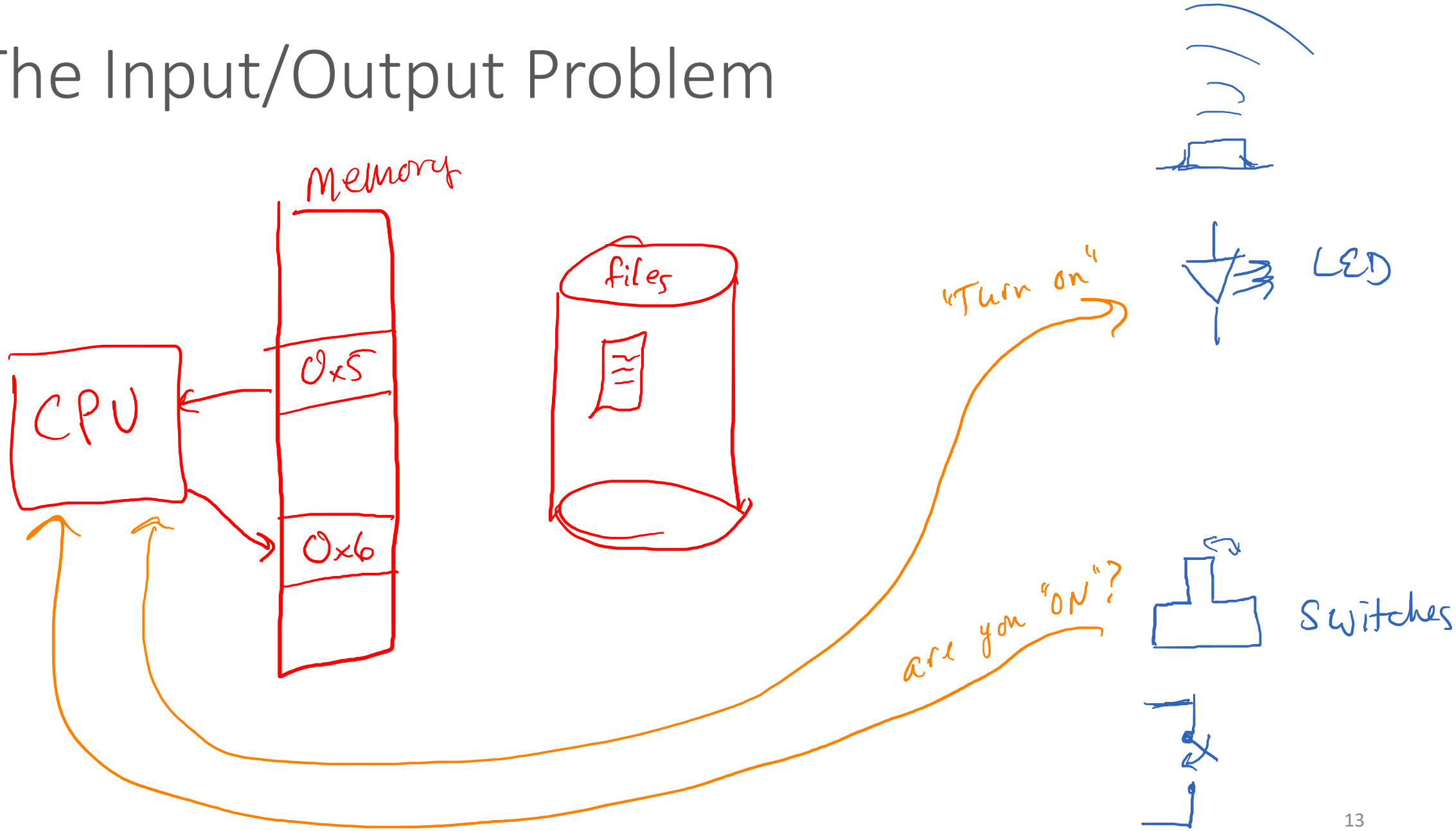
= 0 + S_TDATA[0] + S_TDATA[i]

Popcnt Vivado Demo

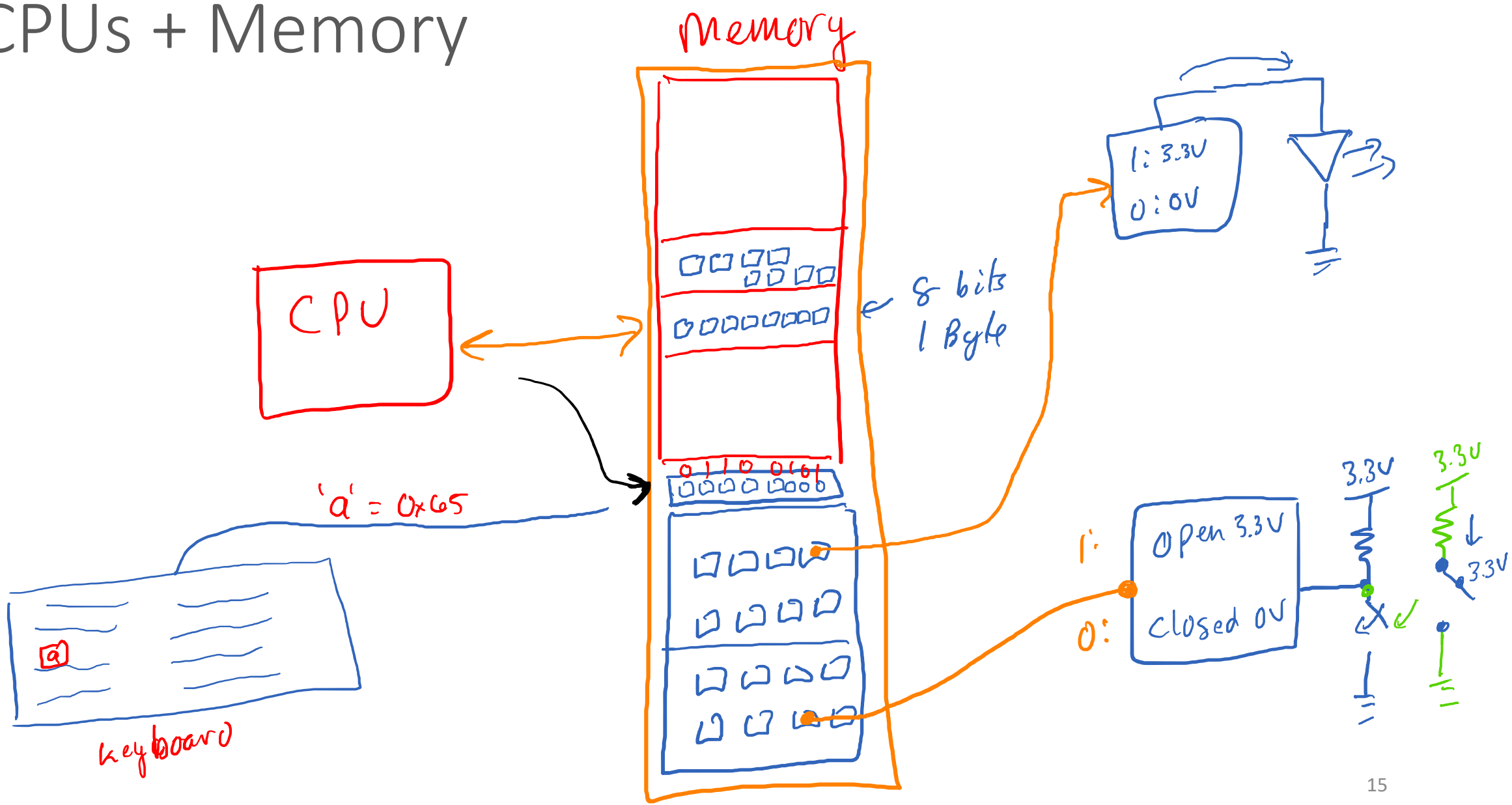
Memory-Mapped Input/Output

(MMIO)

The Input/Output Problem



CPUs + Memory


Memory
CPU
8 bits
1 Byte
'a' = 0x65
keyboard
1: 3.3V
0: 0V
1:
0:
Open 3.3V
Closed 0V
3.3V
3.3V
3.3V

CPUs + Memory + I/O ?

Memory-Mapped I/O

- I/O devices pretend to be memory
- Devices accessed with native CPU load/store instructions

MMIO from Assembly

ARM Registers

R0-R15

ARM	Description	x86
R0	General Purpose	EAX
R1-R5	General Purpose	EBX, ECX, EDX, ESI, EDI
R6-R10	General Purpose	–
R11 (FP)	Frame Pointer	EBP
R12	Intra Procedural Call	–
R13 (SP)	Stack Pointer	ESP
R14 (LR)	Link Register	–
R15 (PC)	<- Program Counter / Instruction Pointer ->	EIP
CPSR	Current Program State Register/Flags	EFLAGS

ARM Instructions

Instruction	Description	Instruction	Description
MOV	Move data	EOR	Bitwise XOR
MVN	Move and negate	LDR	Load
ADD	Addition	STR	Store
SUB	Subtraction	LDM	Load Multiple
MUL	Multiplication	STM	Store Multiple
LSL	Logical Shift Left	PUSH	Push on Stack
LSR	Logical Shift Right	POP	Pop off Stack
ASR	Arithmetic Shift Right	B	Branch
ROR	Rotate Right	BL	Branch with Link
CMP	Compare	BX	Branch and eXchange
AND	Bitwise AND	BLX	Branch with Link and eXchange
ORR	Bitwise OR	SWI/SVC	System Call

ARM Load + Store

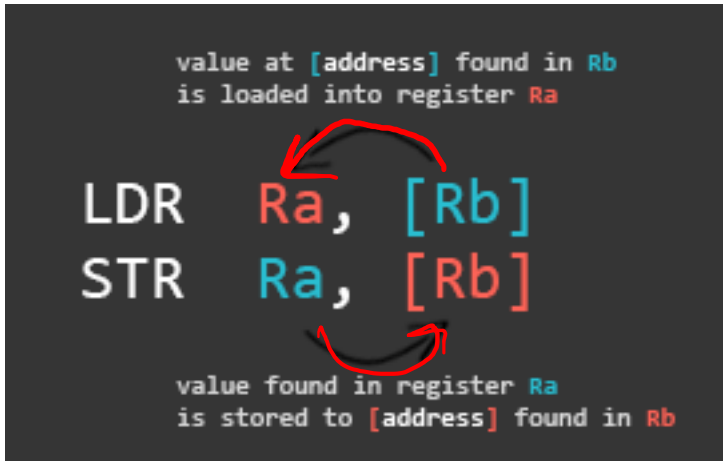
memory of [Address in R0]

LDR R2, [R0] @ [R0] - origin address is the value found in R0.

STR R2, [R1] @ [R1] - destination address is the value found in R1.

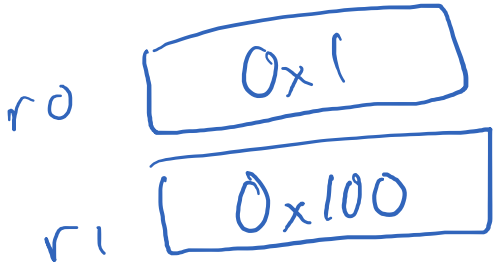
put value of ~~R0~~ R2 into

mem @ address R1

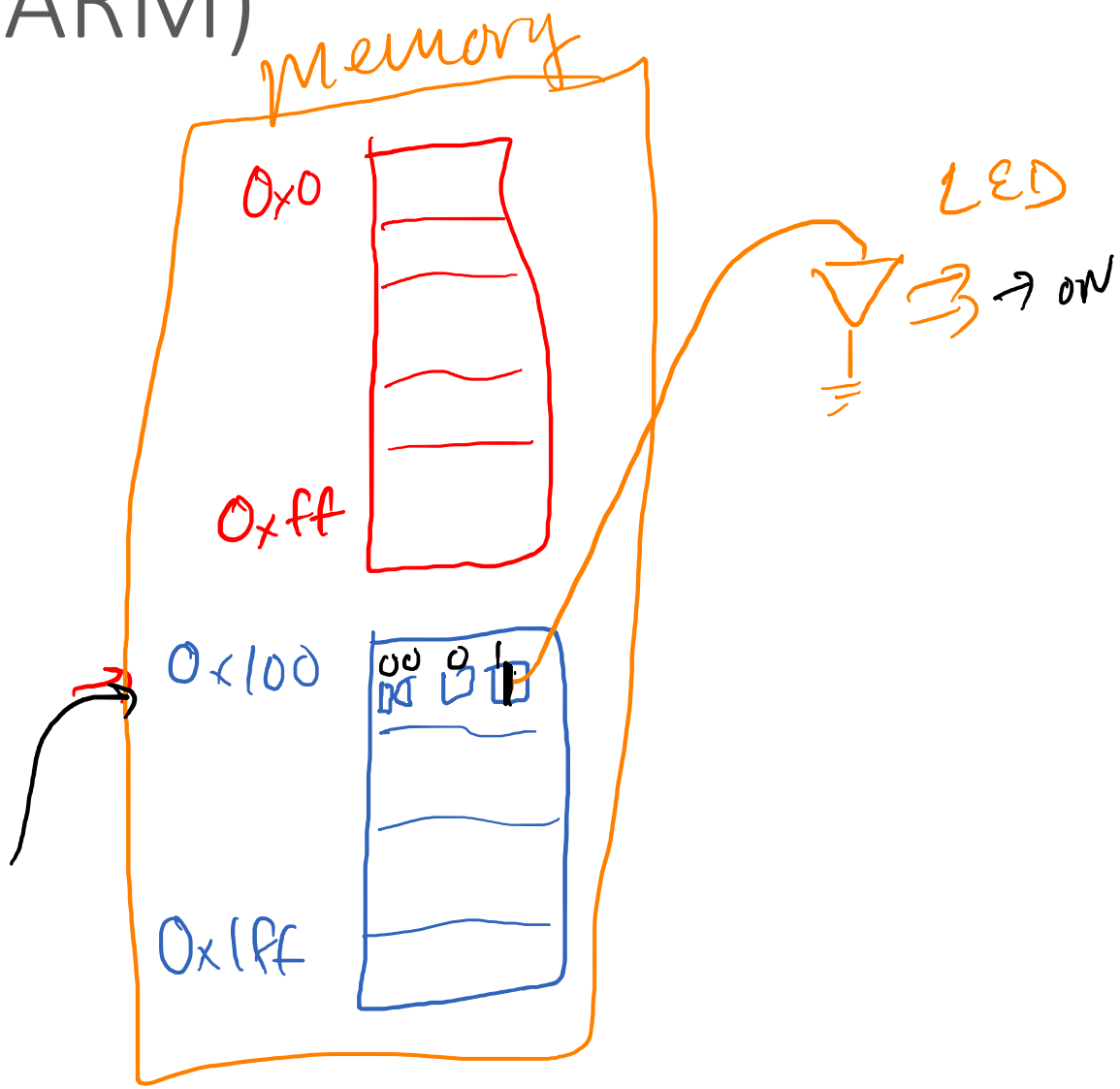


MMIO Store from ASM (ARM)

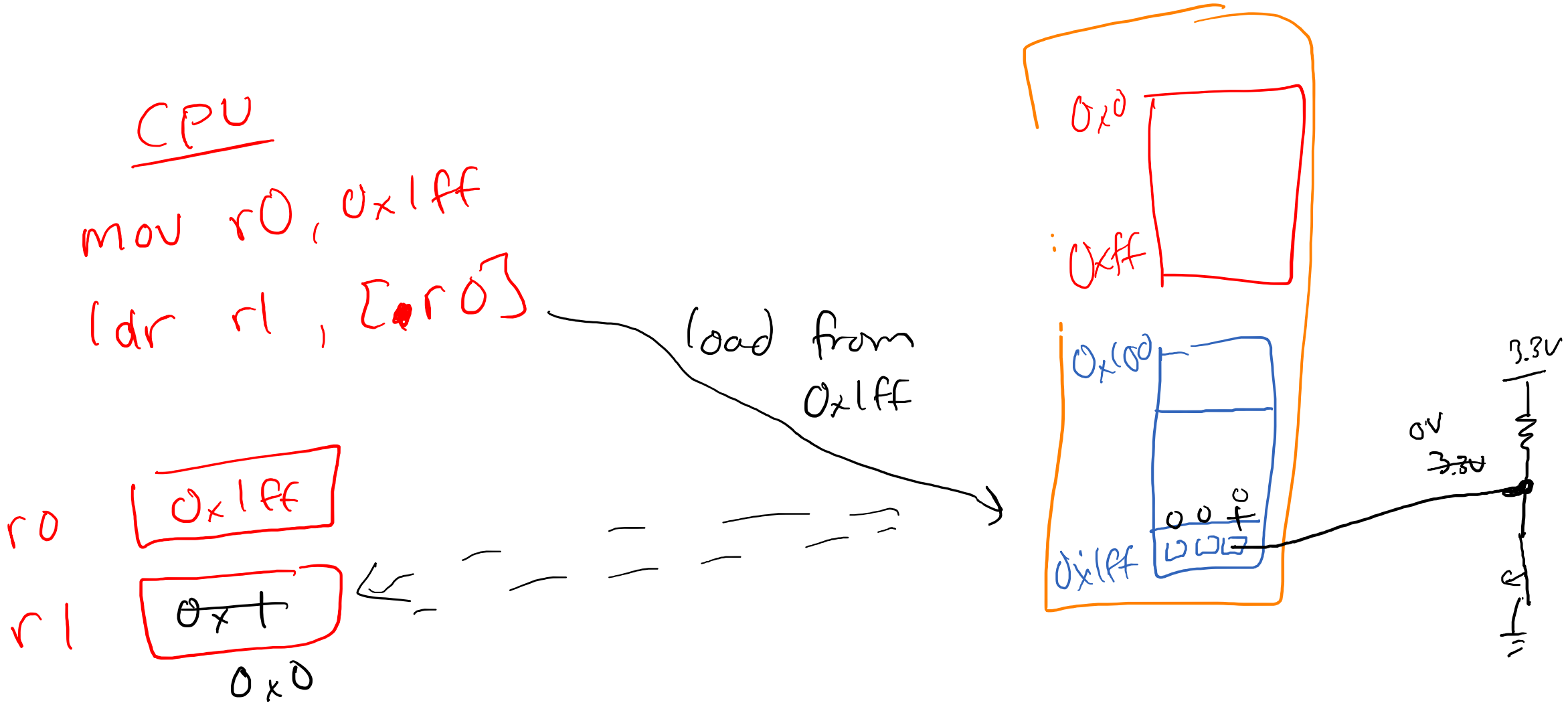
```
mov r0, 0x1
mov r1, [0x100]
str r0, [r1]
```



Store 0x1 to mem[0x100]



MMIO ~~Store~~ load from ~~C~~ ASM



MMIO Store from C

0x100

```
#define LED_ADDR 0xffff
uint32_t * LED_REG = (uint32_t *)(LED_ADDR);
*LED_REG = 0x1;
```

number

(uint32_t *) 0x100

LED_REG = (pointer to 0x100)

* LED_REG = 1

dereference LED_REG → value pointed to by LED_REG → 0x100

set 0x100 = 0x1

MMIO Load from ASM

MMIO Load from ASM

```
mov r1, 0xfffe
ldr r2, [r1]
```

MMIO Load from C

Does `quit` ever change here? Do I need to recompute `(!quit)`?

```
int y = 0;

int quit = y;
while(!quit)
{
    //your code
    quit = y;
}
```

⟹

```
while (1){
    // your code
}
```

Does `quit` ever change here? Do I need to recompute `(!quit)`?

-03

```
int y = 0;
int quit = y;
while(!quit)
{
    //your code
    quit = y;
}
```

// hooked up to a switch

=0 =0 =0 =0 = 1

while (1) {
; ← working
}

skip whole loop

What about here?

```
uint32_t
~~int~~ y = 0;
uint32_t * SW_REG = &y;

int quit = (*SW_REG);
while(!quit)
{
        //your code
        quit = (*SW_REG);
}
```

// optimized out loop

What about here?

```
int y = 0;
uint32_t * SW_REG = &y;

int quit = (*SW_REG);
while(!quit)
{
    //your code
    quit = (*SW_REG);
}
```

constant

constant

constant

⇒

```
while (1) {
    // your code
}
```

Use volatile for MMIO addresses!

```
#define SW_ADDR 0xfffe
volatile uint32_t * SW_REG = (uint32_t * SW_ADDR);

int quit = (*SW_REG);
while(!quit)
{
    //more code
    quit = (*SW_REG);
}
```

`volatile` Variables

- `volatile` keyword tells compiler that the memory value is subject to change randomly.

- Use `volatile` for all MMIO memory. The values change randomly!

Next Time

- Combine MMIO + AXI Bus

Eatai

06: Memory-Mapped I/O

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University